ZOKA QUANTIC

Zero Knowledge by Default, with Post-Quantum Cryptography — Version 2.6 (Design Draft)

Status: Design draft, revised on 30 October 2026 against the source code. **Zoka Quantic** is a blockchain with zero-knowledge proofs and private payments by default. Its private pool is post-quantum by design: spends are proven with hash-based STARKs, notes are encrypted with X-Wing, and nodes and checkpoints sign with ML-DSA-65. It has two separate networks, each with its own genesis: **Zoka Quantic Testnet**, with test coins, which runs this protocol since its v2 genesis on 29 October 2026, with the desktop wallet ZSilent 4.0 and a CLI miner for Windows and Linux; and **Zoka Quantic Mainnet**, coming soon, which launches only after an external review and with a phone wallet. The first testnet chain (testnet v1, 23 to 29 October 2026) ran the earlier Ironwood and Halo 2 design and was replaced. **Zenith**, ZOKA's first network, was the foundation on which we built the node, wallets and networking (section 1); it was retired on 21 October 2026. Nothing from either chain, their balances or their addresses carries over. This document is a proposal, not a security audit.

Status labels used throughout:

Label	Meaning
Implemented	Present in source and tested; does not imply a public binary for every platform.
In development	Being built.
Live	Running on its public network.
Planned	Committed, not started.
Removed	Shipped before, no longer part of ZOKA.
Retired	Ran before; no longer runs.
Dropped	Planned before, no longer needed.

Abstract

Zoka Quantic is a blockchain with zero-knowledge proofs and private payments by default. Every transfer carries one STARK proof, built with Plonky3 over the Poseidon2 hash, showing that the spender owns the notes it consumes, that they are in the commitment tree, that they were not spent before and that inputs and outputs balance. No elliptic curve and no signature take part in a spend. X-Wing combines classical and post-quantum encryption to seal each note for its recipient. Nodes talk over QUIC with a hybrid X25519MLKEM768 handshake and authenticate with ML-DSA-65, and the team's checkpoints carry Ed25519 and ML-DSA-65 signatures. A wallet encrypts each note for its recipient and proves the spend without publishing the input note; the recipient scans and decrypts locally. Nodes validate the rules without needing the private payment contents. Network transport and key custody are part of this design, with their distinct limits stated below. This is what we mean by privacy from end to end; it is not a claim of perfect anonymity against every observer.

A public ledger is permanent. Anyone can store it today and analyse it with whatever computers exist in fifteen years. For a privacy chain, a future quantum computer is therefore not a future problem: what it will be able to read is being written now.

Zoka Quantic builds on the experience of Zenith with a new protocol and chain. Its rule is simple: **post-quantum privacy and validity from the genesis block**. Privacy covers who received a payment, how much, and which note each transaction spent. Validity means that nobody can create coins or spend what is not theirs. Testnet v1 made privacy post-quantum first, with X-Wing, while validity still rested on discrete logarithms. Testnet v2 closes that gap: the soundness of a spend rests on hash functions.

The order was deliberate. A break of privacy reaches backwards over every transaction already on a chain, and nothing can undo it, so privacy had to be post-quantum first. A break of validity can only be exploited once a large quantum computer exists, so it could be closed second — and it had to be closed before the mainnet holds anything of value.

Zoka Quantic does **not** claim to be "quantum-proof". It claims something narrower that can be checked: no part of a spend depends on discrete logarithms. Its security level is an estimate (section 4.1), and the construction awaits an external review.

The design combines:

1. **A STARK pool**: notes are Poseidon2 commitments, nullifiers come from a keyed Poseidon2 function, and one STARK per transaction proves ownership, membership, 64-bit ranges and balance. There is no trusted setup and no spend-authorization signature (section 4.2).
2. **X-Wing note envelopes**: X25519 and ML-KEM-768 (NIST FIPS 203), with ChaCha20-Poly1305 (section 4.3).
3. **Addresses** carrying the owner's digest and an X-Wing key: about 1,760 characters, shared as a 57-character **short address** that the nodes resolve (section 4.4).
4. **ML-DSA-65** (NIST FIPS 204) node identities, bound to each hybrid TLS connection, and hybrid Ed25519 + ML-DSA-65 checkpoints (section 4.7).
5. **A coinbase without a proof and a compact state commitment**: a few kilobytes per block reward and a few hundred bytes of state per block (sections 6 and 7).

Consensus is RandomX proof of work (`rx/0`, which XMRig mines unmodified through a Stratum server built into the node) with a 120-second target, ASERT difficulty and chain selection by accumulated work. The token is **ZKA**, capped at 23,000,000, of which 5,000,000 are allocated to the team at genesis (section 8).

1. Our Foundation and Why a New Chain

Zenith laid the foundation. Our first network, Zenith, is where the node, the wallets, CPU mining and the Tor transport were built and run with real users. It also showed what had to change, and the reasons below are what it taught. Zoka Quantic does not patch Zenith: it is a new protocol and chain built with that experience. Zoka Quantic is the blockchain's identity; a new genesis does not erase where its implementation came from.

From Groth16 to Halo 2 to STARKs. Zenith combined Groth16 zk-SNARKs, commitments and ring-based spending. Its proving parameters came from a single operator, and its transaction format exposed the spend graph (below). Testnet v1 replaced that construction with Ironwood's action circuit and Halo 2, which needs no trusted setup and keeps the consumed note inside the proof, and sealed every note with X-Wing. Privacy became post-quantum; validity did not, because Halo 2 and its spend signatures rest on discrete logarithms. Testnet v2, started from zero on 29 October 2026, is the planned next step: a pool whose proofs rest on hash functions only (section 4.2).

Historical privacy cannot be repaired retroactively. Every output on the Zenith chain was created with classical key agreement. Changing the rules from some block onward would leave every earlier output readable to a future quantum adversary, forever. A new genesis lets every output be born under the new rules.

Zenith published which note each transaction spent. In October 2026 we found that the Zenith private-transaction format carries, in the clear, the commitment and the one-time public key of every note it consumes. An inspection recorded on 11 October 2026 of a 25,204-block local chain snapshot found that its 254 private transactions publish 834 spent notes; only 254 of them sit inside a ring signature, one per transaction; and 559 can be linked directly to the output that created them. Amounts and recipient addresses are not directly published by those fields; off-chain information can still identify participants. The transaction graph is exposed, and no quantum computer is needed to see it. The ring signature does not help, because the real input is published next to the ring. This was a design error of version 1; section 4.2 is how Zoka Quantic removes it.

New formats are cleaner at genesis. Each step changed everything a transaction contains: the proof system, the note format and the addresses. That is simpler on an empty ledger than as a fork of a live one, and every output is born under the same rules.

Two networks, never mixed. Zoka Quantic Testnet and Zoka Quantic Mainnet each have their own chain id, protocol identifier, seeds, ports and data folder, and a node of one cannot join the other. Zenith was retired on 21 October 2026 and testnet v1 on 29 October 2026. Their coins **do not migrate**: there is no balance claim or conversion, and each network starts with its own genesis allocation.

2. Scope and Status

Component	Status
STARK pool transactions: hash-based notes and nullifiers, one proof per transaction for ownership, membership, ranges and balance	Live on the testnet
X-Wing note envelopes, v2 addresses, 57-character short addresses and the nodes' address directory	Live on the testnet
Coinbase without a proof (public value, hidden owner) and compact state commitments	Live on the testnet
XMRig-compatible proof of work (`rx/0`) and a Stratum server in the node	Live; XMRig has mined the testnet since block 1
ASERT difficulty, chain selection by work, timestamp rules	Live
QUIC transport (iroh) with mandatory hybrid TLS and ML-DSA-65 node identities; seeds pinned by key	Live
Hybrid Ed25519 + ML-DSA-65 team checkpoints	Live
Wallet command line (`zoka quantic-pq-wallet`) and the CLI miner for Windows and Linux	Released (4.0.1)
ZSilent 4.0, the official desktop wallet	Released
Zoka Quantic Testnet v2 genesis and first seed	Live since 29 October 2026
Smaller proofs: FRI parameters, aggregation or recursion	In development
Phone wallet (ZKAPriv) on the STARK pool	In development: the pool compiles for ARM; proving on a phone is not measured
Support for Monero pool software (fetch work, submit blocks)	Planned
Shared mining pool without custody (P2Pool-style)	Planned
Code-signed Windows releases	Planned
A relay network of our own for the mainnet	Planned
External review of ZOKA's cryptography and code	Planned, before the mainnet
Zoka Quantic Mainnet genesis	Planned, only after the external review
Multi-party trusted-setup ceremony	Dropped: STARKs have no trusted setup
Tor transport	Removed with testnet v2: the QUIC transport replaces it
Ironwood and Halo 2 transactions (testnet v1)	Retired, 29 October 2026

Component	Status
P2P Exchange (ZKA for USDT)	Removed, October 2026
Zenith, the rehearsal network	Retired, 21 October 2026

No date is committed for the mainnet genesis. Smaller proofs, a phone wallet, device measurements and the external review are prerequisites.

---

3. Threat Model

Zoka Quantic is designed to defend against:

1. A passive observer of the public chain, including anyone indexing block data.
2. **A future observer with a cryptographically relevant quantum computer** analysing the archived chain ("harvest now, decrypt later"). That observer must not learn who received an output, how much a transfer carried, or which note a transaction spent, even if it knows the recipient's address.
3. **The same quantum adversary trying to forge.** It must not be able to create coins or spend outputs it does not own: spends are proven with hash-based STARKs, not with discrete-logarithm assumptions.
4. A node operator who reads all relayed traffic, or who serves blocks to a wallet.
5. An adversary holding the encrypted wallet file without its password.

It does **not** defend against:

1. **A break of its hash assumptions.** Validity rests on Poseidon2 and on the soundness of the STARK (FRI). The security level of section 4.1 is an estimate under the chosen hash assumption, not a proof; a cryptanalytic advance against Poseidon2, or a flaw in ZOKA's circuit, could allow forgery.
2. A global passive adversary watching both ends of every network link in real time.
3. A user who gives the same address to separate off-chain identities. Whoever receives the address can connect those identities off chain. Payments to it remain unlinkable on chain.
4. A compromised recovery phrase or spending key. There is no recovery backdoor.
5. Local disk compromise combined with knowledge of the wallet password.
6. The holder of the team's checkpoint key choosing which valid chain is canonical (section 7.4). That key cannot make an invalid block valid.

---

4. Cryptographic Stack

4.1 Primitives

Primitive	Use	Against a quantum computer
STARK (Plonky3, FRI with zero-knowledge hiding) over the 31-bit KoalaBear field and its degree-4 extension	Spend validity: ownership, membership, nullifiers, 64-bit ranges, balance	Hash-based, no trusted setup. Plonky3's estimator gives 128 bits of classical soundness under the chosen hash assumption; the post-quantum margin is part of the review
Poseidon2 over KoalaBear (width 32)	Note commitments, commitment tree, nullifiers, owner derivation, proof transcript	Hash function: quantum search reduces margins only
X-Wing: X25519 + ML-KEM-768 (FIPS 203), draft revision 10	Note envelopes	Intended to stay secure if either component holds
ChaCha20-Poly1305	Note ciphertexts	Symmetric, 256-bit keys
BLAKE2b and BLAKE3	Address checksums and short addresses, key derivation, transaction digests, state commitment	Quantum search reduces margins only
ML-DSA-65 (FIPS 204)	Node identities, signed into each connection	Post-quantum signature
X25519MLKEM768 (TLS 1.3)	Every connection between nodes	Hybrid: confidential if either part holds
Ed25519 + ML-DSA-65	Team checkpoint signatures	Both required; ML-DSA is the post-quantum component
Argon2 + AES-256-GCM	Wallet file encryption	Password entropy, parameters and implementation remain critical
RandomX	Proof of work	No claim of immunity to quantum mining advantage

These are properties of primitives, not an end-to-end proof of Quantic's privacy or soundness. The estimate above is not a demonstration of 128 bits of quantum security, nor a formal proof of zero knowledge for the whole composition. Ciphertexts, nullifiers, commitments and public proof inputs must be reviewed together.

4.2 STARK Pool Transactions — Live on the testnet

A note and the values derived from it:

```
note   = { owner, rho, rcm, value }          owner = H(OWNER, spending key)
hide   = H(HIDE, owner || rcm)
cm     = H(NOTE, rho || value || hide)
```

```
nf     = H(NULLIFIER, spending key || rho)
```

`H` is Poseidon2 over KoalaBear, with the domain and the network in every message. The commitment `cm` enters the note-commitment tree; the nullifier `nf` is published when the note is spent. Without the spending key, a nullifier cannot be linked to its note.

`rho` **is never chosen.** Output `i` of a transfer takes `rho = H(RHO, first nullifier || i)`, and the circuit enforces it; output `i` of the coinbase at height `h` takes `rho = H(COINBASE_RHO, h || i)`, recomputed by every node. The first nullifier of a transfer is spent once and heights do not repeat, so no two notes of a chain share a `rho`, or therefore a nullifier. An earlier draft let the sender choose `rho`, which would have let a malicious sender make two notes with one nullifier (the "Faerie Gold" attack Zcash fixed after Sprout). The hiding of the owner and the amount comes from `rcm`, random inside `hide`.

A transfer spends 1 to 8 notes and creates 1 to 8. It publishes a **statement**: version, network, an anchor (a recent root of the commitment tree), the nullifiers, the new commitments, the fee and a 32-byte digest of the transaction's other effects, which binds the proof to its envelopes. One STARK proof shows, for the whole transaction, that:

1. the spender knows the spending key behind the owner of every spent note;
2. every spent note is a leaf of the tree under the anchor;
3. every nullifier is correctly derived from its note and that key;
4. every new commitment is well formed, with its `rho` derived from the first nullifier;
5. every value fits in 64 bits, and the spent value equals the created value plus the fee.

Authorization is inside the proof: there is no signature in a spend, so nothing in it rests on an elliptic curve. The proof hides its witness with FRI's zero-knowledge mode.

Measured on a PC: a payment with one input and two outputs is about 1.93 MB and takes about 4.7 s to prove; the largest shape, 8 inputs and 8 outputs, is about 8.7 MB. Proofs dominate the size of every transaction. Section 4.8 covers what that costs.

No trusted setup. A STARK needs no secret parameters. There is nothing to generate in a ceremony and nothing an operator could keep.

4.3 X-Wing Note Envelopes — Live on the testnet

Every output carries an envelope for its recipient. The wallet encapsulates to the recipient's X-Wing key, which travels in the address (1,120-byte ciphertext), derives a symmetric key from the shared secret, and seals the note's opening and a fixed-size 512-byte memo with ChaCha20-Poly1305. The recipient decapsulates with its secret key, decrypts, and accepts the note only if it rebuilds the published commitment. A second ciphertext, sealed with the sender's outgoing key, lets the sender recover what it sent; without that key it reveals nothing.

- Anyone who knows an address can compute nothing from it that opens its notes: the X-Wing secret key never leaves the wallet.
- The hybrid's shared secret is intended to remain secure if either X25519 or ML-KEM-768 holds. The X-Wing specification is a work-in-progress IETF draft; ZOKA pins revision 10 and its test vectors.
- Every memo has the same size, so its length reveals nothing.

4.4 Keys and Addresses — Live on the testnet

Every key derives from the wallet's 24-word recovery phrase, which encodes a 32-byte seed:

```
spending key = derived from (seed, network, account)
view seed    = BLAKE2b("ZokaPQViewKey2__", seed || network || account)
X-Wing key   = the X-Wing key pair generated from the view seed
outgoing key = BLAKE2b("ZokaPQOutKey2___", seed || network || account)
owner        = H(OWNER, spending key)                    16 field elements, 64 bytes
```

The network takes part in every derivation, so the same phrase gives unrelated wallets on each network. An address is:

```
payload  = 0x02 || network || owner (64 bytes) || X-Wing key (1,216 bytes)
checksum = first 4 bytes of BLAKE2b-256("ZokaPQAddress2__", prefix || payload)
address  = prefix || Base58(payload || checksum)                about 1,760 characters
```

What wallets show and share is a **short address**, a fingerprint of the full one:

```
digest = BLAKE2b-256("ZokaPQShort2____", the full address as written)
short  = prefix || Base58(digest || first 4 bytes of BLAKE2b-256("ZokaPQShortSum2_", prefix || digest))
                                                              57 characters on the testnet
```

- A short address carries no key: to pay it, a wallet asks its node for the full address and checks it against the digest before paying. The digest covers every character of the full address, so nobody can keep a real address's visible beginning and swap in their own key.
- **The directory.** Nodes keep a directory of full addresses. A wallet publishes its own with an Argon2id stamp against spam. Each outgoing session pushes the node's own records and the ones it stored recently, at most 30 per minute per connection, and a node asks its peers for a short address that someone is waiting for. The directory is kept by the nodes, not by consensus, and the full address works everywhere.
- The checksum covers the network prefix: a testnet address does not decode as a mainnet one.
- Prefixes: `zq2` (mainnet), `zqtest2` (testnet) and `zqregtest2` (local test networks).
- ZSilent and the CLI show one address per wallet. The address carries the X-Wing key, so whoever receives it can link to it, off chain, the payments they know about.

View keys. A wallet can export a view key (node 4.0.2 and later):

```
payload  = 0x01 || network || owner (64 bytes) || view seed (32) || outgoing key (32)
checksum = first 4 bytes of BLAKE2b-256("ZokaPQViewKeyC2_", prefix || payload)
view key = prefix || Base58(payload || checksum)          zqview2, zqtestview2, zqregtestview2
```

- It opens every note sent to the wallet (the X-Wing key and the owner) and every output of the wallet's own transfers (the outgoing key, which seals all of them). A wallet's balance is therefore exact: what it received, minus the outputs and fee of each transfer it made.
- It **cannot spend**: a nullifier needs the spending key, which the view key does not carry. For the same reason it cannot tell on its own which notes are spent; it infers it from the wallet's transfers.
- Whoever holds it sees every payment of the wallet, past and future. It cannot be revoked; the only remedy is to move the coins to a new wallet.

- **Scanning without telling anyone.** Nodes serve every output of the chain in pages of 100 blocks, without proofs (`/scan`, section 7). ZokaScan opens them with the view key inside the browser, compiled to WebAssembly from the same code as the node, so the server learns that someone downloaded the pages every visitor downloads, and nothing about the wallet.

4.5 Wallet Scanning — Live on the desktop; phones in development

ZSilent and the CLI miner keep a copy of the chain for their wallets, synchronized from the local node over the same authenticated QUIC connection, and validate it like a node. The wallet tries every envelope with its X-Wing key, rebuilds the commitment tree to prove its own notes' membership, and detects its spends by nullifier. Nothing about which notes are the wallet's leaves the computer.

A phone wallet for the STARK pool is in development. The pool library compiles for 32- and 64-bit ARM, but proving has not been measured on a phone: the main low-degree extension of the largest shape alone is estimated at 677 MiB, before other buffers.

4.6 What Testnet v1 Ran — Retired

Testnet v1 used Ironwood transactions through the unmodified `orchard` crate: Halo 2 proofs on the Pasta curves, RedPallas spend-authorization signatures and value commitments on Pallas, with X-Wing envelopes in place of Orchard's key agreement. Its privacy was designed to be post-quantum; its validity was not. Its notes used the recoverable format proposed in ZIP 2005, a possible path for owners if discrete-logarithm validity ever had to be disabled. Testnet v2 replaces the whole spend with the STARK pool, so no note depends on that path, and the mainnet launches with the pool.

4.7 Hybrid Signatures and Node Identities — Live

Checkpoints. The team's chain checkpoints (section 7.4) carry an Ed25519 and an ML-DSA-65 signature over the same domain-separated message, which includes the chain id, and a verifier requires both. Only the public halves of the keys are compiled into the binaries. Testnet v2 reuses the testnet's authority keys; the chain id in the message keeps a signature for one chain from being valid on another. The mainnet will have keys of its own.

Node identities. Every node has an ML-DSA-65 key, stored encrypted. On every connection the serving node signs a message bound to that TLS connection and to the chain, so a signature cannot be replayed on another connection. A seed is pinned by its iroh endpoint id and the BLAKE3 fingerprint of its ML-DSA key. iroh's own endpoint identity is classical (Ed25519); the ML-DSA signature complements it.

Signatures authenticate the publisher; they do not replace block validation.

4.8 Proof Size — In development

A payment is about 1.9 MB, so a block with a few payments weighs megabytes, and storing and relaying the chain costs accordingly. Blocks are also stored as hexadecimal JSON today, twice their binary size. The options are FRI parameters that trade proving time for size, aggregating the proofs of a block, or recursion; Plonky3's recursion has no published release or audit yet. Any of them changes the verifier, so it is a consensus change for a future network, not an optimization of this one. A sustained public network with proofs of several megabytes has not been validated yet.

---

5. What a Quantum Computer Could Do

An adversary with a large quantum computer, reading the archived chain, could…	Zenith	Zoka Quantic Testnet v2
Test whether outputs belong to an address it knows	Yes	Not unless ML-KEM-768 is also broken
Read the amounts and memos of outputs to an address it knows	Yes	Not unless ML-KEM-768 is also broken
Link a spend to the note it consumes	Yes — already exposed without one	No: a nullifier needs the spending key, which is not in the address
Forge proofs or create coins, once the machine exists	Yes	Not unless Poseidon2 or the STARK's soundness fails
Spend others' outputs, once the machine exists	Yes	Same as the row above: authorization is inside the proof

The Zoka Quantic column states the design's hypotheses for the external review, not established results. Quantum search lowers the security margin of every hash function; how much margin the chosen parameters keep is part of the review. Testnet v1 had the first three rows like testnet v2, and the last two like Zenith.

The version 1.0 whitepaper stated that past privacy would survive a quantum break. For Zenith that is not true, and this table replaces it.

---

6. Transaction Format and Validation — Live on the testnet

A transfer publishes its statement (section 4.2), one envelope per output, an optional expiry height and the STARK proof. A transaction can be at most 16 MiB. The transaction id is the BLAKE2b digest of everything but the proof, and the same value is the statement's binding digest, a public input of the proof, so a proof cannot be moved to another transaction.

Every node checks, at mempool admission and again in every block (a miner can include transactions that never entered a local mempool):

1. strict decoding, network and version;
2. shape: 1 to 8 inputs and 1 to 8 outputs, not expired;
3. that the anchor is the commitment-tree root at one of the last 100 blocks of the same branch;
4. that no nullifier was spent before in that branch or repeats within the block;
5. that the fee is at least the conventional fee;
6. the STARK proof.

Coinbase. The first transaction of every block, without a proof. For each output it publishes the `hide` value and the amount, and every node recomputes the commitment with the coinbase `rho` of that height and checks that the amounts add up to exactly the block subsidy plus the block's fees. It is under 4 KB, mostly

the X-Wing envelope for the recipient. The amounts are public; without `rcm`, `hide` reveals no owner, and knowing an address does not let anyone recognize its coinbases. Coinbase notes can be spent 60 blocks later.

Fees are public and uniform. The conventional fee is 0.0001 ZKA for each input or output, counting the larger of the two and at least two (0.0002 ZKA for a simple payment). It is a consensus minimum, so the fee identifies nobody.

The state commitment. Each block commits to the network, height, total issuance, the number of notes, the root of the commitment tree and an accumulator of every nullifier spent:

```
accumulator = BLAKE3(previous accumulator || the block's nullifiers, sorted)
state root  = BLAKE3-XOF-512("ZOKA QUANTIC STARK state v3", committed values)
```

Nodes keep a record of a few hundred bytes per block, of any branch, and validate a branch from those records and the canonical nullifier index. A side branch can therefore spend what the canonical chain spent after the fork and win a reorganization, but never spend the same note twice within itself.

---

7. Consensus and Network

7.1 Proof of Work and Difficulty — Live

ZOKA uses standard RandomX (`rx/0`, the algorithm Monero uses), designed for ordinary CPUs. On Zoka Quantic the hashing blob is 76 bytes, laid out like Monero's, so XMRig and other `rx/0` miners work unmodified:

```
blob  = 0x02 || timestamp (6) || where (32) || nonce (4) || what (32) || 0x00
where = SHA-256("zoka-quantic-pow-v2-where", chain id, previous hash, height)
what  = SHA-256("zoka-quantic-pow-v2-what", Merkle root, state root, extranonce)
```

The work covers the whole block body through the Merkle and state roots. The 64-bit header nonce is `extranonce << 32 | nonce`: a miner changes the 32-bit nonce, and each Stratum connection gets its own extranonce. The hash is compared with the target as a little-endian number, as in Monero, and the RandomX key changes every 2,048 blocks. Nodes only verify hashes, so they use RandomX's light mode (256 MB) and leave the 2 GB dataset to the miner.

Stratum. On the testnet the node does not mine by itself: a miner such as XMRig connects to the Stratum server built into the node (`--stratum host:port`), which speaks the Monero pool protocol. A miner logs in with its own address, full or short, and every block it finds pays it in full; a login that is not an address mines for the node's own wallet. This is solo mining through a node, not a shared pool. The node recomputes every share's hash. XMRig 6.25 has mined testnet v2 since block 1.

- **Target block time:** 120 seconds.
- **Difficulty: ASERT** (`aserti3-2d`, the algorithm Bitcoin Cash adopted in 2020), anchored at block 1, with a one-hour half-life. If blocks arrive faster than the target, the difficulty doubles for every hour the chain runs ahead of schedule; if nobody mines, it falls on its own. Targets are 256-bit numbers stored in Bitcoin's compact format, with a floor of 12 leading zero bits. The testnet starts at 14 bits. Testnet v2 mined its block 1 at launch, so its schedule starts with the chain; it ran ahead during its first hours and

ASERT raised the difficulty accordingly.

- **Chain selection by accumulated work.** Each block counts 2^256 / (target + 1) for the target it had to meet, so the selected chain is the one with the most expected computation.
- **Timestamps** must be later than the median of the previous 11 blocks and no more than two hours ahead of the receiving node's clock.
- Finality is probabilistic. There is no validator set and no stake; see section 7.4 for the team's checkpoints.

The half-life and the initial difficulty will be reviewed on the public testnet before mainnet.

7.2 Transport — Live

Nodes talk over QUIC with **iroh**. Every connection between nodes negotiates X25519MLKEM768 in both directions: a node refuses a classical handshake. The serving node signs each connection with its ML-DSA-65 key (section 4.7). n0's relays and address lookup let nodes behind NAT reach each other and open direct paths, so nobody needs an open port, a public IP address or Tor.

- **Seeds** come from a manifest compiled into the binary, pinned by endpoint id and key fingerprint. Testnet v2's first seed is the team's node.
- **Peer exchange.** A node shares only peers it dialed and authenticated in the last day, up to 16 per answer, and dials and authenticates every peer it learns before storing it. It keeps up to 8 sessions with learned peers and accepts up to 64 inbound.
- **Validation is local.** A node validates every block it receives, whoever serves it, and follows the latest checkpoint before moving blocks.

The relays can see that two endpoints talk, and a direct path reveals IP addresses to the peer. Hybrid TLS protects the contents, not this metadata. The network depends today on n0's public relays; a relay network of our own is planned for the mainnet. The recovery phrase and spending keys stay in the wallet and are never required by peer discovery, a Stratum miner or a block explorer.

No Tor yet. The node reaches its peers over QUIC, which runs on UDP, and Tor only carries TCP, so testnet v2 has no Tor transport: peers and relays see a node's IP address. An optional Tor mode is planned, carrying the node's QUIC packets inside Tor streams to onion seeds, with the hybrid post-quantum TLS still inside (Tor's own handshakes are not post-quantum).

Public read-only API. A node can serve a second, read-only listener for explorers (`--public-api`): status, block summaries, per-block statistics, issued supply, network samples and the `/scan` pages for view keys. GET only, rate-limited, and with nothing that sends, mines or signs. ZokaScan reads one at `api.zokascan.cc`.

7.3 Trusted Setup — None

STARKs have no trusted setup (section 4.2). Zenith's Groth16 parameters were generated by a single operator, which means that operator could in principle forge proofs on Zenith. That concern does not carry over to Zoka Quantic.

7.4 Team Checkpoints — Live; declared trust

The team's node signs checkpoints: "at height H, the canonical block is this hash". It signs every 30 blocks, six blocks below the tip, while the network confirms that tip. Nodes carry the team's public keys compiled in and do not follow any branch that contradicts the latest signed checkpoint, even one with more work. This prevents forks below the checkpoint caused by old or faulty clients, a network partition, or someone with more hash power trying to rewrite history.

This is explicit trust in the team, declared here like the team allocation. The key decides which valid chain is canonical. It cannot make an invalid block valid, create coins outside the emission schedule or spend anyone's funds: every node still validates every block. If the key were lost, the network would continue by accumulated work. If it leaked, an attacker could hold the network on a valid branch of their own until a new binary replaces the key.

---

8. Token Economics — ZKA on Zoka Quantic

Parameter	Value
Symbol	ZKA
Decimals	6 (1 atom = 0.000001 ZKA)
Maximum supply	23,000,000 ZKA
Team allocation at genesis	5,000,000 ZKA (21.7%), spendable from the first block after genesis
Mined supply	18,000,000 ZKA
Initial block reward	12 ZKA
Subsidy reduction	After 1,051,200 mined blocks (about 4 years at 120 s)
Coinbase maturity	60 blocks
Tail emission	None

Emission schedule:

Era	Reward	Blocks	Mined in era	Total supply at end of era
—	Genesis	—	—	5,000,000 (team allocation)
0	12 ZKA	1,051,200	12,614,400 ZKA	17,614,400 ZKA
1	6 ZKA	897,600	5,385,600 ZKA	23,000,000 ZKA

The genesis block pays the team allocation and no subsidy. The next 1,051,200 blocks pay 12 ZKA each, the following 897,600 pay 6 ZKA, and the subsidy is zero afterward, about 7.4 years after genesis at the

target block time. Mining can continue for transaction fees; new coin issuance ends. The node numbers the genesis as block 1, so the subsidy runs from block 2 through block 1,948,801.

On the team allocation. The version 1.0 whitepaper said "no premine, no founder allocation", and Zenith had none. Zoka Quantic has one, and it is stated here plainly: 5,000,000 ZKA to the team, paid in block 1. Like every coinbase, its amount is public and its recipient is hidden. Block 1 itself is fixed in the release binaries by its hash, so a node accepts no other block 1: testnet v2's is `69249437d8c35b7e4bf28ec524491fe18384b9869670fadff546fef29148fdb2`. The allocation is spendable from the next block, with no lock-up and no maturity delay. Spending it is as private as any other spend.

Block rewards are paid to private notes. Every coinbase amount is public, so emission can be audited; the recipient is not.

---

9. Privacy Objectives — Zoka Quantic

9.1 What the Chain Hides

Field	Mechanism
Recipient	X-Wing note envelope (section 4.3); outputs carry no address
Amount of a transfer	Only commitments are public; values are checked inside the proof
Memo	Encrypted inside the envelope, always 512 bytes
Which note a transaction spends	Membership proven inside the STARK (section 4.2)
Sender	No address, key or note of the sender is published

9.2 What the Chain Necessarily Publishes

Field	Reason
Block height, timestamp, proof of work	Consensus
Transaction id; that a transaction occurred; its number of inputs and outputs	Inherent to the format
Expiry height of a transfer, if set	Lets a transfer lapse instead of waiting forever
Nullifiers	Double-spend prevention; unlinkable to notes without the owner's keys
Commitment-tree roots (anchors)	Anchors for the membership proof
Note commitments and envelopes	Needed for validation and for recipients; hiding by construction
The fee	Conventional and uniform (section 6)
Coinbase amounts, including the genesis allocation	So the supply can be audited

---

10. Honest Limits

- **Privacy and validity are hypotheses, not results.** The STARK pool and its circuit, the X-Wing envelopes, the address and transaction formats and the consensus rules are new and not yet externally reviewed.
- **The security level is an estimate.** Plonky3's estimator gives 128 bits of classical soundness under the chosen hash assumption. That is not a proof of post-quantum security, and zero knowledge for the whole composition has not been formally proven.
- **Proofs are large.** About 1.9 MB per payment and up to 8.7 MB for the largest shape (section 4.8). Storage and relaying cost accordingly.
- **The post-quantum libraries are young.** The pure-Rust ML-KEM, ML-DSA and X-Wing crates ZOKA uses are pre-1.0 and, to our knowledge, not independently audited; X-Wing is still an IETF draft; Plonky3 is young too.
- **There is no phone wallet yet.** Proving on a phone has not been measured (section 4.5).
- **Short addresses depend on the directory.** A payer's node has to find the record; until then, only the full address works. A published record shows that the address exists, not its balance or payments.
- **Mining is solo for now.** The Stratum server pays each block to whoever found it. A pool that pays small miners often does not exist yet.
- **Releases are not code-signed yet.** Windows may warn about them; the published SHA-256 checksums are how a download is verified.
- **Some things are public by design:** fees, the number of inputs and outputs, transaction sizes and timing, and every coinbase amount.
- **The network depends on n0's relays** to connect nodes behind NAT, and relays and peers can see IP addresses. There is no Tor path in testnet v2.
- **The team holds a checkpoint key** (section 7.4) **and 21.7% of the supply**, spendable from the start.

- **Zenith's history stays linkable.** Its transaction graph is public (section 1). Moving to Zoka Quantic does not change what Zenith already published.
- **A global passive adversary** watching both ends of every link can correlate timing.
- **There is no recovery.** A lost seed is lost funds, and nobody can reverse a transaction.

---

11. Roadmap

Done:

1. October 2026 security update: desktop Tor and node dependencies updated.
2. Removal of the P2P Exchange.
3. Post-quantum building blocks (ML-KEM-768, X-Wing, ML-DSA-65) and on-device benchmarks.
4. Ironwood with X-Wing, hybrid checkpoints, ASERT and chain selection by work, on testnet v1 (23 October 2026).
5. Short addresses, a standard QR code, XMRig-compatible proof of work and a Stratum server.
6. Zenith retired after serving as the rehearsal; Zoka has two networks, both from zero.
7. The STARK pool: transfer circuit, wallet and node, the coinbase without a proof and the compact state.
8. The QUIC transport with mandatory hybrid TLS and ML-DSA identities, seeds pinned by key, and the team's checkpoints on it.
9. Zoka Quantic Testnet v2 genesis (29 October 2026), with the team's node as the first seed, ZSilent 4.0 and the CLI miner 4.0 for Windows and Linux.

Next, in order, without dates:

1. Smaller proofs: FRI parameters, aggregation or recursion.
2. The phone wallet on the STARK pool, with measurements on devices.
3. Support for Monero pool software, so existing pools can add Zoka.
4. Code-signed releases and a relay network of our own.
5. External review.
6. Zoka Quantic Mainnet genesis.
7. After the mainnet: a shared mining pool without custody, in the style of P2Pool.

---

12. Reference Code

Source repositories are hosted on GitHub at github.com/zoka-network and GitLab at gitlab.com/zokanetwork. Projects include the node (`zokanetwork`, which also holds the CLI miner in `packaging/cli-miner`), the desktop wallet (`zsilent-core`), the Android wallet (`zkapriv`), the web explorer of Zenith, the rehearsal network (`zokaexplorer`), and this site (`zokaweb`). The STARK pool is `crates/zoka-pq-pool`; the node that runs it is `src/quantic/stark*.rs`, built with the `quantic-pq` feature, and its commands are `zoka quantic-pq-node` and `zoka quantic-pq-wallet`. Testnet v1's transaction library remains in `crates/zoka-quantic`. Releases are verified with their published SHA-256 checksums.

Primary references:

- NIST FIPS 203: ML-KEM.

- NIST FIPS 204: ML-DSA.
- X-Wing, revision 10 (work in progress).
- Plonky3.
- Poseidon2: A Faster Version of the Poseidon Hash Function.
- ASERT difficulty adjustment (aserti3-2d).
- iroh.

These references describe primitives and constructions; none audits ZOKA.

---

13. Closing Position

ZOKA exists because the right to transact in private is the last frontier of individual freedom. Version 1.0 promised the strongest privacy that implementation knew how to ship, with its limits in plain text. Version 2 keeps that promise and adds what we learned since: privacy has to be measured against the computers of the future, and against our own mistakes. Both are written down here.

Verify what is true. Reveal nothing else.

— Natosh Zoka, core dev @ ZokaNetwork